

Technical Assessment

LATAM E-commerce Scraping Team — Fintech

Estimated time: Part 1 ~5–8h, Part 2 ~5–8h

Delivery window: 5 business days

Implementation language: Python

PART 1

Item Condition Classifier

Predict whether a Mercado Libre listing is new or used

1. Context

In the context of Mercado Libre’s Marketplace, an algorithm is needed to predict whether an item listed in the marketplace is new or used. This is a real signal the platform uses to improve search quality, ranking, fraud detection, and buyer trust. We want to evaluate how you approach a binary classification problem end-to-end: from inspecting and understanding a real, messy marketplace dataset, through feature engineering and model selection, to evaluating results honestly. We care about your reasoning at every step at least as much as the final score.

2. The Task

Build a machine learning solution that predicts whether a marketplace item is new or used, and evaluate the model on held-out test data.

2.1 Dataset

- **Dataset file:** `MLA_100k.jsonlines` (provided alongside this assessment).
- **Loader:** a starter file `new_or_used.py` is provided with a `build_dataset` function. Use it as your entry point for reading the data.
- **Target variable:** the `condition` field of each listing (new vs. used).

2.2 Hard requirements

1. Use `build_dataset` from `new_or_used.py` to load the data. Do not load the file in a different way.

2. Perform exploratory data analysis on the dataset and document your findings (class balance, feature distributions, missing values, leakage risks, etc.).
3. Design the feature processing pipeline. Justify every non-trivial choice (encoding, imputation, text handling, dropping columns, etc.).
4. Train and evaluate a model that achieves at least 0.86 accuracy on the held-out test split returned by `build_dataset`.
5. Choose an appropriate secondary metric (in addition to accuracy) and write a short argument explaining why you chose it for this specific problem.
6. Provide reproducible code: a fixed random seed, deterministic data splits, and clear instructions to reproduce the reported numbers.

2.3 Bonus points (not required)

- Compare at least two model families and explain the trade-offs.
- Add basic error analysis: where does the model fail, and what does that tell you about the data?
- Add a small notebook that prints metrics and a confusion matrix when run.

3. Deliverables

1. Source code in a Github repo, containing your data processing, training, and evaluation code.
2. A short report (Markdown, notebook, or PDF) covering: dataset overview, feature decisions, model selection rationale, final metrics on the test set, your secondary metric and the argument behind it, limitations, and what you would do with more time.
3. Reproducibility: requirements file (`pyproject.toml`), a one-command way to run training and evaluation, and the random seed used.

4. Notes

- You may use any ML library (scikit-learn, XGBoost, LightGBM, PyTorch, etc.).
- You may use AI assistants for coding, but be ready to defend every modeling decision in the follow-up interview.
- Do not spend more than ~8 hours of focused work on this part. If you need to cut, document what you cut and why.

PART 2

Multi-Agent Scraping System

Orchestrate scraping agents across LATAM e-commerce sites

1. Context

You are joining a fintech team that builds infrastructure to gather pricing, product, and checkout/payment-method information from LATAM e-commerce sites. The data feeds downstream products that need it to be timely, structured, and trustworthy.

This part of the assessment focuses on your ability to design and ship an agentic scraping solution. We are evaluating a Tech Lead, so we care less about a feature-perfect implementation and more about how you reason about the problem, how you structure code, and the engineering trade-offs you make explicit.

What we want to learn from this exercise

- How you decompose an open-ended problem into agents with clear responsibilities.
- How you handle the messy reality of e-commerce sites: dynamic content, anti-bot measures, A/B variants, regional differences.
- Your judgment on when to use an LLM vs. deterministic code, and how you control cost and latency.
- Your code quality, testing approach, and operational thinking (logging, observability, error handling).

2. The Task

Build a Python service that exposes an API endpoint. Given a product URL from a LATAM e-commerce site, the system orchestrates at least two cooperating agents to navigate from the product page to the checkout and extract the available payment methods.

2.1 Hard requirements

1. Implementation must be in Python. No other languages for the core solution.
2. Use an agent framework of your choice: LangGraph, CrewAI, Google ADK, smolagents, AutoGen, or any other you prefer.
3. At least two agents must be orchestrated. They must have distinct, well-defined responsibilities, not just be the same agent renamed twice.

4. Use an LLM provider of your choice: Anthropic Claude, OpenAI, Google Gemini, Ollama (local or cloud), Mistral, etc
5. Expose an HTTP API (FastAPI, Flask, etc.) with at least one endpoint that accepts a URL and returns a structured result.
6. The system must handle at least two different LATAM e-commerce sites. Suggested targets: Falabella, Linio, Shopify stores, TiendaNube stores, Woocommers stores, Amazon MX/BR, Magalu, Americanas, Liverpool, Ripley, Exito. You may pick others.
7. Return structured output (JSON) including, at minimum, the list of payment methods detected at checkout and the source URL.
8. Include a README with setup instructions, an architecture diagram (any format: ASCII, Mermaid, image), and a section explaining your design decisions and trade-offs.

2.2 Bonus points (not required)

- Extract product title and price during the navigation, and include them in the response.
- Detect installment plans ("cuotas", "meses sin intereses", "parcelado") and surface them as structured data.
- Handle currency normalization across LATAM countries (COP, MXN, BRL, ARS, CLP, PEN).
- Add a /health endpoint and structured logging.
- Containerize with Docker.
- Add a small evaluation harness (e.g. 5–10 sample URLs and expected fields) so we can run it ourselves.

2.3 Out of scope

- You do NOT need to actually complete a purchase. Reaching the screen where payment methods are displayed (checkout selector, payment options modal, or equivalent) is enough.
- You do NOT need to handle authentication / login flows. You may pick products that allow guest checkout, or stop at the payment options preview that most sites show pre-login.
- You do NOT need to bypass CAPTCHAs or sophisticated anti-bot systems. If you hit one, return a clean error and document it.
- You do NOT need a UI. The deliverable is the API.

3. API Contract

We are giving you a suggested contract. You may extend it or propose a new structure

Endpoint

POST /scrape

Request body

JSON

```
{
  "url": "https://www.mercadolibre.com.co/...",
  "country": "CO",
  "options": {
    "extract_title": true,
    "extract_price": true,
    "timeout_seconds": 60
  }
}
```

Successful response (200)

JSON

```
{
  "status": "ok",
  "source_url": "https://www.mercadolibre.com.co/...",
  "site": "mercadolibre",
  "product": {
    "title": "Apple iPhone 15 128GB",
    "price": {
      "amount": 4999000,
      "currency": "COP"
    }
  },
  "payment_methods": [
    {
```

```
    "type": "credit_card",
    "brand": "Visa",
    "installments": {
      "max": 36,
      "interest_free_max": 12
    }
  },
  {
    "type": "credit_card",
    "brand": "Mastercard",
    "installments": {
      "max": 36,
      "interest_free_max": 12
    }
  },
  {
    "type": "debit_card",
    "brand": "Visa Debit"
  },
  {
    "type": "wallet",
    "brand": "Mercado Pago"
  },
  {
    "type": "bank_transfer",
    "brand": "PSE"
  },
  {
    "type": "cash",
    "brand": "Efecty"
  }
],
"metadata": {
  "duration_ms": 18432,
```

```
"agent_steps": 7,
"llm_calls": 3,
"llm_tokens": {
  "input": 4200,
  "output": 380
}
}
```

Error response (4xx / 5xx)

JSON

```
{
  "status": "error",
  "source_url": "https://...",
  "error": {
    "code": "CHECKOUT_UNREACHABLE",
    "message": "Reached cart but checkout button required login.",
    "stage": "navigator"
  }
}
```

Define your own error codes; we want to see that you thought about failure modes (timeouts, captchas, geo-blocks, out-of-stock items, anti-bot detection).

4. Deliverables

1. A GitHub repository with the full source code. Public repo is fine.
2. A README.md at the root with: project description, prerequisites, setup steps, how to run the API locally, example curl request, architecture overview, design decisions, known limitations.
3. A pyproject.toml (uv / poetry).
4. A .env.example file listing the environment variables required (API keys, model names, etc). Do NOT commit real keys.

5. At least 3 example product URLs you tested, and their expected output (in the README or a /examples folder).
6. Optional but appreciated: a Dockerfile and docker-compose.yml so we can run it with one command.

How we run it

We will clone your repo, follow your README, and run something like:

Shell

```
# 1. install:
uv sync # or: poetry install

# 2. set keys:
cp .env.example .env && edit .env

# 3. install browsers (if Playwright):
playwright install chromium

# 4. run:
uvicorn app.main:app --reload

# 5. Test:
curl -X POST http://localhost:8000/scrape \
  -H 'Content-Type: application/json' \
  -d '{"url": "https://www.store-name.com.co/..."}'
```

If our setup deviates from yours, we will follow your README. So please make sure it is accurate. If something requires a specific OS, model access, or paid API key we may not have, document alternatives (e.g. "works with both Claude and a local Ollama model").

5. Ground Rules

- **Use AI assistants freely.** We expect you to use Copilot, Cursor, Claude Code, or similar. We care about the result and your understanding of it, not whether you typed every character.
- **Be ready to defend every decision.** In the follow-up interview we will ask you to walk through the code, change parts of it live, and reason about edge cases. "The AI wrote it" is not an acceptable answer.
- **Respect the sites you scrape.** Use reasonable rate limits, identify yourself sanely, do not hammer endpoints. We are not asking you to evade defenses.

- **Do not spend more than 8 hours of focused work.** If you go over, document what you cut. We would rather see a smaller, polished system than a large unfinished one.

Follow-up Interview

After review, we will schedule a 60 minute session. The format is roughly:

- 10 min: you walk us through the architecture and the key decisions.
- 30 min: live code reading. We pick a couple of files and ask you to explain, defend, or refactor.
- 15 min: extension questions.
- 5 min: your questions for us.

Good luck. We are looking forward to seeing what you build.